21
SCREAMING ARCHITECTURE

Imagine that you are looking at the blueprints of a building. This document, prepared by an architect, provides the plans for the building. What do these plans tell you?

If the plans you are viewing are for a single-family residence, then you'll likely see a front entrance, a foyer leading to a living room, and perhaps a dining room. There will likely be a kitchen a short distance away, close to the dining room. Perhaps there is a dinette area next to the kitchen, and probably a family room close to that. When you looked at those plans, there would be no question that you were looking at a single family home. The architecture would scream: "HOME."

Now suppose you were looking at the architecture of a library. You would likely see a grand entrance, an area for check-in/out clerks, reading areas, small conference rooms, and gallery after gallery capable of holding bookshelves for all the books in

the library. That architecture would scream: “LIBRARY.”

So what does the architecture of your application scream? When you look at the top-level directory structure, and the source files in the highest-level package, do they scream “Health Care System,” or “Accounting System,” or “Inventory Management System”? Or do they scream “Rails,” or “Spring/Hibernate,” or “ASP”?

THE THEME OF AN ARCHITECTURE

Go back and read Ivar Jacobson’s seminal work on software architecture: *Object Oriented Software Engineering*. Notice the subtitle of the book: *A Use Case Driven Approach*. In this book Jacobson makes the point that software architectures are structures that support the use cases of the system. Just as the plans for a house or a library scream about the use cases of those buildings, so should the architecture of a software application scream about the use cases of the application.

Architectures are not (or should not be) about frameworks. Architectures should not be supplied by frameworks. Frameworks are tools to be used, not architectures to be conformed to. If your architecture is based on frameworks, then it cannot be based on your use cases.

THE PURPOSE OF AN ARCHITECTURE

Good architectures are centered on use cases so that architects can safely describe the structures that support those use cases without committing to frameworks, tools, and environments. Again, consider the plans for a house. The first concern of the architect is to make sure that the house is usable—not to ensure that the house is made of bricks. Indeed, the architect takes pains to ensure that the homeowner can make decisions about the exterior material (bricks, stone, or cedar) later, after the plans ensure that the use cases are met.

A good software architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed. *Frameworks are options to be left open.* A good architecture makes it unnecessary to decide on Rails, or Spring, or Hibernate, or Tomcat, or MySQL, until much later in the project. A good architecture makes it easy to change your mind about those decisions, too. A good architecture emphasizes the use cases and decouples them from peripheral concerns.

BUT WHAT ABOUT THE WEB?

Is the web an architecture? Does the fact that your system is delivered on the web dictate the architecture of your system? Of course not! The web is a delivery mechanism—an IO device—and your application architecture should treat it as such. The fact that your application is delivered over the web is a detail and should not dominate your system structure. Indeed, the decision that your application will be delivered over the web is one that you should defer. Your system architecture should be as ignorant as possible about how it will be delivered. You should be able to deliver it as a console app, or a web app, or a thick client app, or even a web service app, without undue complication or change to the fundamental architecture.

FRAMEWORKS ARE TOOLS, NOT WAYS OF LIFE

Frameworks can be very powerful and very useful. Framework authors often believe very deeply in their frameworks. The examples they write for how to use their frameworks are told from the point of view of a true believer. Other authors who write about the framework also tend to be disciples of the true belief. They show you the way to use the framework. Often they assume an all-encompassing, all-pervading, let-the-framework-do-everything position.

> *This is not the position you want to take.*

Look at each framework with a jaded eye. View it skeptically. Yes, it might help, but at what cost? Ask yourself how you should use it, and how you should protect yourself from it. Think about how you can preserve the use-case emphasis of your architecture. Develop a strategy that prevents the framework from taking over that architecture.

TESTABLE ARCHITECTURES

If your system architecture is all about the use cases, and if you have kept your frameworks at arm’s length, then you should be able to unit-test all those use cases without any of the frameworks in place. You shouldn’t need the web server running to run your tests. You shouldn’t need the database connected to run your tests. Your Entity objects should be plain old objects that have no dependencies on frameworks

or databases or other complications. Your use case objects should coordinate your Entity objects. Finally, all of them together should be testable in situ, without any of the complications of frameworks.

CONCLUSION

Your architecture should tell readers about the system, not about the frameworks you used in your system. If you are building a health care system, then when new programmers look at the source repository, their first impression should be, “Oh, this is a heath care system.” Those new programmers should be able to learn all the use cases of the system, yet still not know how the system is delivered. They may come to you and say:

“We see some things that look like models—but where are the views and controllers?”

And you should respond:

“Oh, those are details that needn’t concern us at the moment. We’ll decide about them later.”